UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

INGENIERÍA EN COMPUTACIÓN

ARQUITECTURA DE COMPUTADORAS

MIPS

(Tipo R)

Nombres:

- Luis Giovanni Rodríguez Hernandez
- Sara Iriel Alonso Jaquez
- Leonel Octavio Rios Loreto
- Jorge Abraham Moran Encina

Fecha: 24 de Abril de 2026

1. Introducción

El desarrollo de un procesador es uno de los aspectos más interesantes y esenciales en el ámbito de la ingeniería informática. Para comprender el funcionamiento interno de una computadora, es crucial primero conocer cómo se crean los circuitos digitales que llevan a cabo las instrucciones.
En esta tarea, se elabora el DataPath Tipo R (DPTR), que representa la primera versión operativa, aunque simplificada, de un procesador MIPS. Un DataPath es, en esencia, el trayecto que recorre la información dentro del procesador: desde el momento en que se recibe una instrucción, hasta que los datos son procesados y el resultado es almacenado.
Las instrucciones que puede ejecutar este DataPath son del tipo R, lo que significa que son operaciones aritméticas y lógicas que operan únicamente con registros. Las operaciones que se pueden realizar son:

- ADD – Realiza la suma de dos registros.
- SUB – Efectúa la resta de dos registros.
- AND – Lleva a cabo una operación AND a nivel de bits.
- OR – Realiza una operación OR a nivel de bits.
- SLT – Set on Less Than (asigna un 1 si el primer operando es menor que el segundo).

Para lograr esto, se combinan módulos que han sido previamente diseñados (ALU, Banco de Registros, Memoria) con nuevos componentes: la Unidad de Control, el Control de ALU y un Multiplexor 2:1. Todo se interconecta en el módulo principal DPTR, que funciona como un procesador completo en versión reducida.

Resumen del video:
En el video explica la arquitectura MIPS, en la manera de cómo se construye y componentes del procesador, además de los tipos de instrucciones y funcionamiento, donde hay muchos tipos de instrucciones, pero solo usaremos las instrucciones más sencillas o básicas. En el video se centra más en el formato de instrucciones de Tipo R que son de registros, siendo de 32 bits en el que se divide en varios espacios para identificar los registros de entrada (RS, RT), registro de destino (RD)y el campo de función(funct), el cual determina la operación que se realizará en la ALU.
Se explica que el código de operación (Opcode) es siempre 0, y es el campo de función el que le indica a la ALU que operación realizar (suma, resta, AND, etc.). También se menciona la diferencia de arquitectura MIPS y la x86 (utilizada por Intel y AMD) donde la ventaja que el MIPS al ser arquitectura RISC, maneja tres registros (dos de entrada y un destino) lo que simplifica el diseño del hardware.
Además, pero cla dificultad de este procesador MIPS no está tanto en los módulos individuales, si no en la interconexión entre ellos. También muestra un ejemplo con una resta donde se observa el flujo completo, como los datos salen del Banco de Registros, pasan por la ALU y el resultado regresa para almacenarse en el registro de destino.

Formato de instrucción tipo R

Una instrucción MIPS tipo R tiene 32 bits de ancho y está dividida en 6 campos:

Campo	Bits	Tamaño	Descripción
OpCode	31:26	6 bits	Código de operación (siempre 000000 para tipo R)
RS	25:21	5 bits	Registro fuente 1
RT	20:16	5 bits	Registro fuente 2
RD	15:11	5 bits	Registro destino
Shamt	10:6	5 bits	Desplazamiento (no usado en estas instrucciones)
Function	5:0	6 bits	Código de función que especifica la operación

La separación de estos campos no es física en el hardware: la instrucción llega como 32 bits y cada módulo toma los bits que le corresponden según su posición.

2. Marco Teórico

Procesador MIPS y Arquitectura RISC

MIPS (Microprocessor without Interlocked Pipeline Stages) es una arquitectura de procesador de tipo RISC (Reduced Instruction Set Computer). Esto significa que utiliza un conjunto reducido de instrucciones, todas del mismo ancho (32 bits), lo que simplifica enormemente el diseño del hardware.

A diferencia de las arquitecturas CISC (como x86), donde las instrucciones pueden tener longitudes variables y ser muy complejas, en MIPS cada instrucción hace exactamente una cosa simple. Esto permite que el procesador sea más rápido y eficiente.

Instrucciones Tipo R

Las instrucciones tipo R son las instrucciones aritméticas y lógicas de MIPS. Se llaman "tipo R" porque todos sus operandos provienen de Registros. Su OpCode siempre es 000000 (6 bits en cero), y la operación específica se distingue por el campo Function (bits 5:0).

Los códigos de función para las instrucciones implementadas son:

Instrucción	OpCode	Función (binario)	Función (hex)	Descripción
ADD	000000	100000	0x20	Suma con signo: RD = RS + RT
SUB	000000	100010	0x22	Resta con signo: RD = RS - RT
AND	000000	100100	0x24	AND bit a bit: RD = RS & RT
OR	000000	100101	0x25	OR bit a bit: RD = RS | RT
SLT	000000	101010	0x2A	Set Less Than: RD = 1 si RS < RT
NOR	000000	100111	0x27	NOR bit a bit: RD = ~(RS | RT)

La instruccion SLT (Set on Less Than)

La instrucción SLT merece una explicación especial. SLT compara dos registros y coloca en el registro destino el valor 1 si el primer operando (RS) es menor que el segundo (RT), o 0 en caso contrario. Formalmente:

RD = (RS < RT) ? 1 : 0

Esta instrucción es la base de las estructuras condicionales en MIPS. Para poder implementarla en la ALU, se necesita usar el operador ternario en Verilog, que tiene la forma:

resultado = (condición) ? valor_si_verdadero : valor_si_falso;

Por ejemplo, para que la ALU realice SLT: R = (A < B) ? 32'd1 : 32'd0; Esto significa: "si A es menor que B, el resultado es 1; de lo contrario es 0".

El operador ternario en Verilog

El operador ternario es una forma compacta de escribir una condición if-else en una sola línea. Su sintaxis en Verilog es:

assign resultado = (condición) ? valor_verdadero : valor_falso;

Es ampliamente usado dentro de bloques always y en asignaciones continuas. Su principal ventaja es que simplifica el código cuando la lógica es simple. En el contexto de la ALU, permite implementar operaciones como SLT sin necesidad de estructuras if-else más largas.

Proceso de Compilación

Cuando se escribe un programa en un lenguaje de alto nivel (como C o C++), este pasa por varias etapas antes de convertirse en código que el procesador puede ejecutar:

- Preprocesamiento: Se procesan las directivas del compilador (como #include o #define). El resultado es un archivo de código fuente "limpio".
- Compilación: El compilador traduce el código fuente a lenguaje ensamblador (Assembly). En este paso se hacen optimizaciones y se detectan errores de sintaxis.
- Ensamblado: El ensamblador convierte el código Assembly en código máquina binario (archivos objeto .o). Cada instrucción Assembly se traduce a su equivalente en bits.
- Enlazado (Linking): El enlazador une todos los archivos objeto y las bibliotecas necesarias para producir el ejecutable final.

En el contexto de MIPS, las instrucciones tipo R que se implementan en este proyecto son justamente el resultado de ese proceso de ensamblado: cada instrucción de alto nivel como a + b termina siendo una instrucción ADD en código máquina con sus 32 bits en el formato tipo R descrito.

Descripción de los módulos del DataPath

- Banco de Registros

El Banco de Registros es una memoria interna del procesador que contiene 32 registros de 32 bits cada uno. En una instrucción tipo R se leen dos registros (RS y RT) como operandos y se escribe el resultado en un tercer registro (RD). En la implementación actual, los registros están inicializados con valores del 10 al 20 para facilitar las pruebas.

- ALU (Unidad Aritmético-Lógica)

La ALU realiza la operación matemática o lógica indicada. Recibe dos operandos de 32 bits y una señal de selección de 4 bits (SelAlu) que le indica qué operación debe realizar. En este proyecto la ALU debe soportar ADD, SUB, AND, OR, SLT y NOR.

- Unidad de Control

La Unidad de Control recibe el OpCode de la instrucción (6 bits) y genera las señales de control para todos los demás módulos. Para instrucciones tipo R (OpCode = 000000), activa RegWrite (para escribir el resultado en el banco de registros), desactiva MemWrite y MemRead, y envía AluOp = 010 para indicar que la ALU Control debe consultar el campo Function.

- ALU Control

La ALU Control recibe la señal AluOp de la Unidad de Control y el campo Function de la instrucción. Si AluOp indica una instrucción tipo R, decodifica el campo Function para determinar qué señal de selección enviar a la ALU. Es el módulo que traduce el código de función en la operación concreta de la ALU.

- Multiplexor 2:1 (Mux2a1)

El multiplexor selecciona entre dos entradas de 32 bits y dirige una de ellas a la salida según la señal de control UCctl. Si UCctl = 0, la salida es el resultado de la ALU; si UCctl = 1, la salida es el dato leído de la memoria de datos. Esto permite que el banco de registros reciba datos tanto del resultado de la ALU como de la memoria.

- Memoria de Instrucciones (MEMI)

La Memoria de Instrucciones almacena las instrucciones del programa. Recibe una dirección y entrega la instrucción de 32 bits que reside en esa dirección. Se inicializa leyendo un archivo de texto (inst.txt) con el código máquina en formato binario, usando la directiva $readmemb de Verilog.

- Memoria de Datos (MEMA)

La Memoria de Datos almacena los datos del programa. Para instrucciones tipo R no se utiliza directamente, pero es parte del DataPath completo. Soporta operaciones de lectura (RE) y escritura (WE) controladas por las señales de la Unidad de Control.

3. Objetivo

El propósito principal de esta actividad es crear e implementar en Verilog un DataPath de ciclo único que sea capaz de ejecutar instrucciones MIPS de tipo R. Para alcanzar este objetivo, se unen los módulos que ya se han desarrollado, como la ALU, el Banco de Registros, la Memoria de Datos y la Memoria de Instrucciones, junto con los nuevos módulos necesarios para esta fase del proyecto. Además, se llevará a cabo la implementación de la Unidad de Control, cuya función será generar las señales de control adecuadas basándose en el OpCode de cada instrucción R, así como la ALU Control, que tendrá como tarea decodificar el campo Function para determinar la operación adecuada dentro de la ALU.

De igual manera, se creó un Multiplexor 2:1 que permite elegir el dato que será guardado en el Banco de Registros, siendo este el resultado que provee la ALU o el dato que llega de la memoria. A continuación, todos estos módulos se enlazan dentro del módulo principal DPTR, con el fin de formar el DataPath completo. Finalmente, se simula el TestBench (TB_DPTR) para comprobar el correcto funcionamiento del sistema, enviando instrucciones tipo R que estén codificadas en formato binario de 32 bits. Con esto, se busca entender de manera completa el proceso de una instrucción tipo R dentro del DataPath, desde su ingreso como una secuencia de bits hasta que el resultado final es registrado en el Banco de Registros.

4. Desarrollo

Unidad de Control – UNITCTRL.v

La Unidad de Control recibe el campo OpCode de 6 bits de la instrucción y genera todas las señales de control del sistema. Para instrucciones tipo R el OpCode es siempre 000000.

```
module UNITCTRL (
    input [5:0]Op,
    output reg Regwrite,
    output reg MemWrite,
    output reg MemRead,
    output reg MemToReg,
    output reg [2:0]AluOp
);
    always @(Op) begin
        case (Op)
            6'b000000 : begin
                Regwrite  =  1'b1;     //  Habilita  escritura  en  el  banco  de
registros
                MemWrite = 1'b0;   // No escribe en memoria de datos
                MemRead  = 1'b0;   // No lee de memoria de datos
                MemToReg = 1'b0;   // El dato a guardar viene de la ALU
                AluOp    = 3'b010; // Indica a ALUCtrl que consulte el campo
Function
            end
        endcase
    end
endmodule
```

La señal AluOp con valor 010 le indica a la ALU Control que la instrucción es tipo R y que debe usar el campo Function para determinar la operación. La señal RegWrite en 1 habilita que el resultado se guarde en el registro RD.

ALU Control – ALUCTRL.v

La ALU Control recibe AluOp y el campo Function y determina el valor de SelAlu (4 bits) que se envía a la ALU. Para AluOp = 010 (instrucción tipo R) decodifica el campo Function:

```
module ALUCTRL (
    input [2:0]AluOp,
    input [5:0]func,
    output reg[3:0]SelAlu
);
    always @(AluOp) begin
        case (AluOp)
            3'b010 : begin
                case (func)
                    6'b100000 : SelAlu = 4'b0010; // ADD
                    6'b100010 : SelAlu = 4'b0110; // SUB
                    6'b100100 : SelAlu = 4'b0000; // AND
                    6'b100101 : SelAlu = 4'b0001; // OR
                    6'b101010 : SelAlu = 4'b0111; // SLT
                    6'b100111 : SelAlu = 4'b1100; // NOR
                endcase
            end
        endcase
```

```
    end
endmodule
```

La tabla de decodificación resume la correspondencia entre el código de función y la operación de la ALU:

Instrucción	func (binario)	SelAlu (binario)	SelAlu (decimal)	Operación en ALU
ADD	100000	0010	2	Suma
SUB	100010	0110	6	Resta
AND	100100	0000	0	AND bit a bit
OR	100101	0001	1	OR bit a bit
SLT	101010	0111	7	Menor que (ternario)
NOR	100111	1100	12	NOR bit a bit

Multiplexor 2:1 – Mux2a1.v

El Mux selecciona qué dato se escribe en el Banco de Registros: el resultado de la ALU (cuando UCctl = 0) o el dato proveniente de la Memoria de Datos (cuando UCctl = 1). Para instrucciones tipo R siempre se selecciona el resultado de la ALU.

```
module Mux2a1 (
    input [31:0] mem, alu,
    input UCctl,
    output reg[31:0] R
);
    always @(*)
        case(UCctl)
            1'd0: R <= alu;  // Instrucciones tipo R: usa resultado de ALU
            1'd1: R <= mem;  // Instrucciones load: usa dato de memoria
        endcase
endmodule
```

Banco de Registros – BR.v

El Banco de Registros almacena 32 registros de 32 bits. Lee dos registros simultáneamente (AR1 y AR2) y escribe en uno (AW) cuando RegEn está en alto. Los primeros 10 registros están inicializados con valores del 10 al 20 para facilitar la verificación de las pruebas.

```
module BR (
    input RegEn,
    input [4:0]AR1, AR2, AW,
    input [31:0]DW,
    output reg[31:0]DR1, DR2
);
    reg [31:0]mem[0:31];
    initial begin
        mem[0]=32'd10; mem[1]=32'd11; // ... mem[10]=32'd20;
    end
    always @* begin
        DR1 = mem[AR1];
        DR2 = mem[AR2];
        if (RegEn) mem[AW] = DW;
    end
endmodule
```

Memoria de Instrucciones – MEMI.v

La Memoria de Instrucciones lee instrucciones de un archivo de texto en formato binario. Implementa Big Endian: toma 4 bytes consecutivos y los concatena para formar la instrucción de 32 bits.

```
module MEMI (
    input [31:0]dir,
    output reg[31:0]DSalida
);
    reg[7:0] MEM[0:255];
    initial begin
        $readmemb("inst.txt", MEM); // Carga instrucciones desde archivo
    end
    always @(dir) begin // Big Endian: byte más significativo primero
        DSalida <= {MEM[dir], MEM[dir+1], MEM[dir+2], MEM[dir+3]};
    end
endmodule
```

Módulo Integrador – DPTR.v

El módulo DPTR conecta todos los componentes formando el DataPath completo. Las señales internas (wires) llevan la información entre módulos siguiendo el flujo de la instrucción:

```
module DPTR (
    input [31:0]InstTR  // Instrucción de 32 bits entrante
);
    // Señales internas (cables)
    wire [31:0]C1;  // DR2: segundo operando del banco de registros
    wire [31:0]C2;  // DR1: primer operando del banco de registros
    wire [31:0]C3;  // Resultado de la ALU
    wire [3:0]C4;   // Señal SelAlu hacia la ALU
    wire [31:0]C5;  // Salida de la Memoria de Datos
    wire [31:0]C6;  // Salida del Mux (dato a escribir en registro)
    wire CRW, CMW, CMR, CMTR; // Señales de control
    wire [2:0]CA;   // AluOp de la Unidad de Control

    // Instancias de los módulos
    BR br(.RegEn(CRW), .AR1(InstTR[25:21]), .AR2(InstTR[20:16]),
           .AW(InstTR[15:11]), .DW(C6), .DR1(C2), .DR2(C1));
    ALUMIPS alu(.A(C2), .B(C1), .Sel(C4), .R(C3));
    MEMA mem(.DIR(C3), .WE(CMW), .RE(CMR), .DATA_IN(C1), .DATA_OUT(C5));
    Mux2a1 mux(.mem(C5), .alu(C3), .UCctl(CMTR), .R(C6));
    UNITCTRL uc(.Op(InstTR[31:26]), .Regwrite(CRW), .MemWrite(CMW),
                .MemRead(CMR), .MemToReg(CMTR), .AluOp(CA));
    ALUCTRL aluctrl(.AluOp(CA), .func(InstTR[5:0]), .SelAlu(C4));
Endmodule
```

Flujo de Ejecución de una Instrucción Tipo R

Para entender cómo funciona el DataPath, se puede seguir el recorrido de una instrucción ADD paso a paso. Supongamos la instrucción: add $8, $0, $1 (sumar el registro 0 y el registro 1, guardar en el registro 8).

1. La instrucción de 32 bits entra al módulo DPTR a través de InstTR. Los bits [31:26] son el OpCode (000000).
2. La Unidad de Control recibe el OpCode y activa las señales: RegWrite=1, MemWrite=0, MemRead=0, MemToReg=0, AluOp=010.

3. El Banco de Registros lee DR1 (registro RS, bits [25:21]) y DR2 (registro RT, bits [20:16]) simultáneamente.
4. La ALU Control recibe AluOp=010 y el campo Function (bits [5:0]). Para ADD (100000), genera SelAlu=0010.
5. La ALU recibe los dos operandos y SelAlu=0010, realiza la suma y entrega el resultado en C3.
6. El Mux recibe MemToReg=0, por lo que selecciona la salida de la ALU (C3) y la pasa a C6.
7. El Banco de Registros, con RegWrite=1, escribe el valor de C6 en el registro RD (bits [15:11]).

TestBench – TB_DPTR.v

El TestBench conecta el DPTR con la Memoria de Instrucciones y suministra direcciones para que se ejecuten instrucciones reales cargadas desde inst.txt. La instrucción almacenada en la dirección 0 se decodifica como sigue:

```
``timescale 1ns/1ns
module TB_DPTR;
wire [31:0]InstTR;
reg [31:0]DIR;
DPTR mydptr(.InstTR(InstTR));
MEMI mymemi(.dir(DIR), .DSalida(InstTR));
initial begin
    #100
    #100 DIR <= 32'd0;
    #100 DIR <= 32'd4;
    #100 DIR <= 32'd8;
    #100 DIR <= 32'd12;
    #100 DIR <= 32'd16;
    #100 DIR <= 32'd20;
    #100
    $finish;
end
endmodule
```

Tabla de Instrucciones de prueba

A continuación, se presenta una tabla con las 6 instrucciones de prueba (una de cada tipo), en formato ensamblador y su equivalente en código máquina de 32 bits. Se asume que los registros $0 al $11 contienen valores del 10 al 21 respectivamente (según la inicialización del Banco de Registros).

#	Ensamblador	OpCode	RS	RT	RD	Shamt	Func	Resultado Esperado
1	add $13,$0,$1	000000	00000	00001	01101	00000	100000	$13 = 21
2	sub $14,$2,$3	000000	00010	00011	01110	00000	100010	$14 = -1
3	and $15,$4,$5	000000	00100	00101	01111	00000	100100	$15 = 14
4	or $16,$6,$7	000000	00110	00111	10000	00000	100101	$16 = 17
5	slt $17,$8,$9	000000	01000	01001	10001	00000	101010	$17 = 1
6	nor $18,$10,$11	000000	01010	01011	10010	00000	100111	$18 = -22

5. Conclusión

Crear el DataPath Tipo R desde el principio, conectando cada módulo individualmente, permite visualizar claramente cómo una instrucción de 32 bits se traduce en una acción real dentro del hardware.

Uno de los aprendizajes clave fue entender cómo actúan la Unidad de Control y la ALU Control como módulos "inteligentes" que decodifican los bits de la instrucción y producen las señales necesarias para que cada parte del sistema sepa cómo actuar. Sin estas unidades, el DataPath sería simplemente una colección de piezas desconectadas.

La realización de la instrucción SLT fue particularmente educativa, ya que implicó ajustar la ALU para incorporar el operador ternario de Verilog, que resulta ser una herramienta sumamente útil para expresar lógica condicional de manera concisa. Esto demuestra cómo el diseño de hardware puede utilizar características del lenguaje de descripción para facilitar las implementaciones.

La combinación de todos los módulos en DPTR, junto con la comprobación a través del TestBench, validó que el diseño opera de manera efectiva para las instrucciones tipo R. Observar en la simulación que los registros del Banco de Registros se actualizan con los resultados correctos de la ALU proporciona evidencia concreta de que el procesador reducido está funcionando.

Por último, vincular este proyecto con el proceso de compilación nos ayuda a entender que las instrucciones tipo R son el resultado de transformar código de alto nivel a código máquina, ofrece una visión más amplia y da sentido al trabajo de diseño de hardware dentro del ciclo completo del software.

6. Referencias

Patterson, D. A., & Hennessy, J. L. (2014). Computer Organization and Design: The Hardware/Software Interface (5.ª ed.). Morgan Kaufmann Publishers.

Ciletti, M. D. (2011). Advanced Digital Design with the Verilog HDL (2.ª ed.). Pearson/Prentice Hall.

MIPS Technologies. (2005). MIPS32 Architecture for Programmers, Volume II: The MIPS32 Instruction Set. MIPS Technologies Inc.

Harris, D. M., & Harris, S. L. (2012). Digital Design and Computer Architecture (2.ª ed.). Morgan Kaufmann.